Original lecture content:
TOPIC 3
SYNTAX ANALYSIS
From tokens and grammar rules to deterministic parsing and syntax trees.

Original lecture content:
INTRODUCTION
Learning roadmap
Recognise what a parser does and where it fits in a compiler.
Describe context-free grammars, syntax trees and derivations.
Explain leftmost/rightmost derivation and ambiguity.
Build a predictive LL(1) parser using a single running example.
Contrast top-down and bottom-up parsing.
CPT316 • Programming Language Implementation and Paradigms

Original lecture content:
INTRODUCTION
Why syntax analysis matters
Input
Parser
Output
Tokens from the lexical
analyser
Checks token order
against grammar rules
Parse structure or a
syntax error
Syntax analysis establishes structure; semantic analysis checks its meaning.

Diagram representation / semantic relationships:
Input: tokens from lexical analyser -> Parser: checks token order against grammar rules -> Output: parse structure OR syntax error.

Original lecture content:
INTRODUCTION
The compiler pipeline
Source code
Characters
→
Scanner
Tokens
→
Parser
Syntax structure
→
Semantic analysis
Meaning + types
The symbol table is shared by compiler phases to store identifiers and their properties.

Diagram representation / semantic relationships:
Source code (characters) -> Scanner (tokens) -> Parser (syntax structure) -> Semantic analysis (meaning + types). Symbol table is shared by compiler phases.

Original lecture content:
INTRODUCTION
From tokens to grammatical structure
x = y + z * 60
Scanner: id = id + id * number
Parser: assignment with an expression on the right
AST: multiplication binds more tightly than addition

Original lecture content:
GRAMMAR FOUNDATIONS
GRAMMARS & TREES
Understand grammar notation before constructing parse trees
and derivations.
CPT316 • TOPIC 3

Original lecture content:
GRAMMARS & TREES
Abstract syntax tree (AST)
For x = y + z * 60
=
id(x)
+
id(y)
*
id(z)
The tree groups z * 60 before applying +, then assigns the result to x.
CPT316 • Programming Language Implementation and Paradigms

Diagram representation / semantic relationships:
AST for x = y + z * 60: root (=); left child id(x); right child (+); (+) left child id(y), right child (*); (*) children id(z) and 60. Expression: x = y + (z * 60).

Original lecture content:
GRAMMARS & TREES
Parse tree vs. abstract syntax tree
Parse tree (concrete syntax)
Abstract syntax tree (AST)
Includes grammar nonterminals,
terminals and possible ε-productions.
Useful for showing how a grammar
derives an input.
Keeps the essential operators and
operands. Often omits grammar-
specific nodes and punctuation.
a parse tree records derivation; an AST records essential structure.

Original lecture content:
GRAMMARS & TREES
Context-free grammar: four components
Symbol
Meaning
Example
V (or N)
Nonterminals / syntactic categories
{E, T, F}
Σ (or T)
Terminals / tokens
{id, +, *, (,)}
P
Production rules
E → E + T | T
S
Start symbol
E
A context-free production has exactly one nonterminal on its left-hand side.
CPT316 • Programming Language Implementation and Paradigms

Diagram representation / semantic relationships:
CFG table columns Symbol | Meaning | Example. V (or N) | nonterminals | {E,T,F}; Sigma (or T) | terminals | {id,+,*,(,)}; P | productions | E -> E+T | T; S | start symbol | E.

Original lecture content:
GRAMMARS & TREES
Anatomy of a production rule
A → a B b | ε
Left-hand side
Right-hand side
Exactly one nonterminal: A
A string of terminals and/or nonterminals, or ε
(empty string).
CPT316 • Programming Language Implementation and Paradigms

Diagram representation / semantic relationships:
Production A -> a B b | epsilon. Left-hand side A is exactly one nonterminal. Right-hand side is a sequence of terminals and nonterminals or the empty string.

Original lecture content:
GRAMMARS & TREES
Example: is A → aBb a valid rule?
Grammar class
Valid?
Reason
Context-free
Linear
Right-linear
Left-linear
“Linear” is broader than “right-linear” or “left-linear” they are not synonyms.

Original lecture content:
GRAMMARS & TREES
Right-linear and left-linear grammars
Right-linear
Left-linear
A → wB or A → w
A → Bw or A → w
The optional nonterminal is at the right
end.
The optional nonterminal is at the left
end.
Here w is a string of terminals. Each type can generate regular languages.

Original lecture content:
GRAMMARS & TREES
Example: does babab belong to the grammar?
S → TaTaT
T → aT | bT | ε
Question
Can S derive the string babab?
Identify the two required a tokens and
what each T derives.

Original lecture content:
GRAMMARS & TREES
Parse trees: the grammar made visible
Root node: the start nonterminal.
Internal nodes: nonterminals expanded using productions.
Leaves: terminals (and ε when explicitly shown).
Reading terminal leaves left to right gives the derived string.
One parse tree has one leftmost and one rightmost derivation; the rewriting order can differ.

Original lecture content:
GRAMMARS & TREES
Worked example: parse tree for bbabb
Grammar: S → bSb | a | b
S
b
S
b
b
S
b
a
Left-to-right leaves: b b a b b. Expand S → bSb twice, then S → a.
CPT316 • Programming Language Implementation and Paradigms

Diagram representation / semantic relationships:
Parse tree for bbabb: root S -> b S b; middle S -> b S b; innermost S -> a. Leaves in order b b a b b.

Original lecture content:
GRAMMARS & TREES
Practice: can the string be derived?
S → AB | ε
A → aB
B → Sb
Test both strings
aabbbb
aabbb

Original lecture content:
GRAMMARS & TREES
Leftmost versus rightmost derivation
Leftmost derivation
Rightmost derivation
At every step, rewrite the leftmost
remaining nonterminal.
At every step, rewrite the rightmost
remaining nonterminal.
Both can describe the same parse tree. A different order alone does NOT prove ambiguity.

Original lecture content:
GRAMMARS & TREES
Class example: derive a + b * c
E → E + E | E * E | a | b | c
Construct one parse tree that interprets the expression as a + (b * c).
Write its leftmost derivation and its rightmost derivation.
Then compare with the alternative tree for (a + b) * c.

Original lecture content:
GRAMMARS & TREES
Homework: derivations and parse trees
S → T00T
T → 0T | 1T | ε
Input
1000111
Choose one valid derivation, then show
its leftmost and rightmost forms.
Hint: the two fixed 0 tokens can occur at more than one position in some strings.

Original lecture content:
GRAMMARS & TREES
When is a grammar ambiguous?
A grammar is ambiguous if some string has two different parse trees.
Equivalently, some string has two distinct leftmost derivations.
This can lead to different operator groupings or other interpretations.
Different rewriting orders of one tree are not evidence of ambiguity.
CPT316 • Programming Language Implementation and Paradigms

Original lecture content:
GRAMMARS & TREES
Class example: demonstrate ambiguity
S → A B A
A → aA | ε
B → bB | ε
String: aa
One tree derives both a symbols from
the first A. Another derives both from
the final A.
Draw the two trees.

Original lecture content:
GRAMMARS & TREES
Homework: ambiguity in expression grammar
E → E + E | E * E | id
For id + id * id, draw two structurally different parse trees.
Label the interpretation of each: (id + id) * id vs id + (id * id).
Explain why ordinary arithmetic precedence is not encoded by this grammar.

Original lecture content:
TOP-DOWN AND BOTTOM-UP
PARSING STRATEGIES
Connect derivation order with how practical parsers are built.
CPT316 • TOPIC 3

Original lecture content:
PARSING STRATEGIES
Top-down vs. bottom-up parsing
Top-down: root → leaves
Bottom-up: tokens → root
Starts with the start symbol and predicts productions.
Starts from input tokens and repeatedly reduces handles.
Usually constructs a leftmost derivation.
Corresponds to a rightmost derivation in reverse.
Examples: recursive descent, LL(1).
Examples: shift-reduce, LR, SLR, LALR.
CPT316 • Programming Language Implementation and Paradigms

Diagram representation / semantic relationships:
Top-down: start from start symbol and expand toward leaves. Bottom-up: shift tokens and reduce substrings to nonterminals until start symbol.

Original lecture content:
TOP-DOWN PARSING
Top-down example: expression grammar
E → T E′
E′ → + T E′ | ε
T → F T′
T′ → * F T′ | ε
F → (E) | id
Input: id + id * id. The grammar forces multiplication to bind more tightly than addition.

Original lecture content:
TOP-DOWN PARSING
Top-down parse tree: id + id * id
E
T
E′
F
T′
+
T
E′
id
ε
F
T′
ε
id
*
F
T′
id
ε
Observe where the * belongs: inside the second T subtree.

Diagram representation / semantic relationships:
[UNVERIFIED VISUAL RELATIONSHIP: Exact parse-tree branch layout for id + id * id requires manual inspection; the derivation follows grammar expansions.]

Original lecture content:
TOP-DOWN PARSING
Recursive-descent parser: idea
Create one procedure for each nonterminal.
On a terminal, match the expected token and advance.
For alternatives, use lookahead to decide which rule applies.
Predictive recursive descent uses lookahead and does not backtrack.
Accept only when the start procedure finishes AND the input is exhausted.

Original lecture content:
TOP-DOWN PARSING
Recursive descent: E, T and F
procedure E():
procedure F():
if lookahead == '(':
T(); EPrime()
match('('); E();
match(')')
procedure T():
else if lookahead == id:
F(); TPrime()
match(id)
else: error()
match(t) reports an error unless the next input token is exactly t.

Original lecture content:
TOP-DOWN PARSING
Recursive descent: E′ and T′
procedure EPrime():
procedure TPrime():
if lookahead == '+':
if lookahead == '*':
match('+'); T(); EPrime()
else if lookahead in {')',
'$'}:
match('*'); F(); TPrime()
else if lookahead in {'+',
')', '$'}:
return // ε
else: error()
return // ε
else: error()
An ε-branch is legal only when the lookahead belongs to that nonterminal’s FOLLOW set.

Original lecture content:
TOP-DOWN PARSING
Homework: recursive descent and parse tree
S → if E then S else S | begin L end | print E
L → S ; L | ε
E → num = num
Task
Construct a parse tree for: begin print num = num ; end
Identify each use of L → ε.

Original lecture content:
LL(1) WORKFLOW
PREDICTIVE PARSING
One expression grammar, six connected steps, one completed parse.

Original lecture content:
PREDICTIVE PARSING
What does LL(1) mean?
First L
Second L
Read input Left to
right.
Produce a Leftmost
derivation.
Use One lookahead
token to choose the
production.
A predictive LL(1) parser makes its choice without backtracking.

Original lecture content:
PREDICTIVE PARSING
LL(1) parser components
Stack
Input
Table
Output
Grammar symbols, $
at bottom
Tokens followed by $
M[nonterminal,
lookahead]
Productions / tree /
errors
At each step: match a terminal, expand a nonterminal using the table, or report an error.

Original lecture content:
PREDICTIVE PARSING
Running example: one grammar, six steps
E → E + T | T
T → T * F | F
F → (E) | id
Target input
id + id * id
Start symbol: E
Input marker: $
We will transform this grammar and use it throughout FIRST, FOLLOW, table building and parsing.

Original lecture content:
PREDICTIVE PARSING
Predictive parsing roadmap
STEP 1 OF 6
Remove left recursion
Completed / earlier
Current focus
Next
—
Remove left recursion
Check left factoring
Focus: rewrite the grammar without changing its language.
CPT316 • Programming Language Implementation and Paradigms

Diagram representation / semantic relationships:
Six-step LL(1) roadmap: remove left recursion -> left factor -> FIRST/FOLLOW -> parsing table -> parsing trace -> parse tree.

Original lecture content:
PREDICTIVE PARSING
Recognising recursion in a grammar
Direct left recursion
Right recursion
A → Aα | β
A → αA | β
The recursive call follows material that can
consume input (when α begins with a
terminal).
The right-hand side begins with A. A
predictive top-down parser may loop.
Indirect left recursion is also possible: A ⇒+ Aα through one or more other nonterminals.

Original lecture content:
PREDICTIVE PARSING
Why direct left recursion causes a loop
A → Aa | b
Incorrect substitution: A →
aA | b
A() calls A() first —
before consuming any
input.
This changes ba, baa, ...
into ab, aab, ...
Use a language-preserving transformation, not a simple swap of A and a.

Original lecture content:
PREDICTIVE PARSING
Eliminating direct left recursion: rule 1
Before
After
A → Aα | β
A → βA′
A′ → αA′ | ε
Assume β does not begin with A.
Example: A → Aa | b transforms into A → bA′, A′ → aA′ | ε.
CPT316 • Programming Language Implementation and Paradigms

Diagram representation / semantic relationships:
Direct left recursion rule: A -> A alpha | beta becomes A -> beta A_prime and A_prime -> alpha A_prime | epsilon.

Original lecture content:
PREDICTIVE PARSING
Practice: eliminate direct left recursion
E → E + T | T
T → T * F | F
F → id
Your task
Find A, α and β for each left-recursive rule, then write the transformed grammar.

Original lecture content:
PREDICTIVE PARSING
Running example – Step 1
Rule
α
β
Replacement
E → E + T | T
T → T * F | F
F → (E) | id
New grammar.

Original lecture content:
PREDICTIVE PARSING
Direct left recursion: several α alternatives
$A \rightarrow A\alpha_1 \mid A\alpha_2 \mid \beta$
$A \rightarrow \beta A'$
$A' \rightarrow \alpha_1 A' \mid \alpha_2 A' \mid \varepsilon$
The recursive suffixes become alternatives of the new A′ nonterminal.
CPT316 • Programming Language Implementation and Paradigms

Original lecture content:
PREDICTIVE PARSING
Worked practice: two nonterminals
A → ABd | Aa | a
B → Be | b

Original lecture content:
PREDICTIVE PARSING
Direct left recursion: multiple β alternatives
$A \rightarrow A\alpha_1 \mid A\alpha_2 \mid \beta_1 \mid \beta_2$
$A \rightarrow \beta_1 A' \mid \beta_2 A'$
$A' \rightarrow \alpha_1 A' \mid \alpha_2 A' \mid \varepsilon$
All non-left-recursive β alternatives remain alternatives of A, each followed by A′.

Original lecture content:
PREDICTIVE PARSING
Class exercise: combine multiple cases
A → Ad | Ae | aB | ac
B → bBc | f

Original lecture content:
PREDICTIVE PARSING
Indirect left recursion
A → Ba | b
B → Ac | d

Original lecture content:
PREDICTIVE PARSING
How to remove indirect left recursion
Choose a fixed order A_1, A_2, …, A_n.
For A_i, substitute alternatives beginning with earlier A_j ($j < i$).
Then eliminate any direct left recursion in A_i.
Repeat until every nonterminal has been processed.
Example result: $A \to bA' \mid daA'$; $A' \to caA' \mid \varepsilon$

Original lecture content:
PREDICTIVE PARSING
Predictive parsing roadmap
STEP 2 OF 6
Check left factoring
Completed / earlier
Current focus
Next
Remove left recursion
Check left factoring
Compute FIRST + FOLLOW
Next: check whether alternatives share a prefix.

Original lecture content:
PREDICTIVE PARSING
Left factoring: why it matters
A → ab | ac

Original lecture content:
PREDICTIVE PARSING
Left factoring in one step
$A \to \alpha\beta_1 \mid \alpha\beta_2 \mid \gamma$
$A \to \alpha A' \mid \gamma$
$A' \to \beta_1 \mid \beta_2$
Find the longest common prefix α.
Move the different suffixes into a new nonterminal A′.
CPT316 • Programming Language Implementation and Paradigms

Diagram representation / semantic relationships:
Left factoring: A -> alpha beta1 | alpha beta2 becomes A -> alpha A_prime and A_prime -> beta1 | beta2.

Original lecture content:
PREDICTIVE PARSING
Running example – Step 2
Nonterminal
Alternatives
Common prefix?
E
E′
T
T′
F

Original lecture content:
PREDICTIVE PARSING
Class exercise: left factoring
S → aSSbS | aSaSb | abb | b
Identify the two longest shared prefixes carefully.
Introduce new nonterminals where necessary.
Check that every original alternative is still represented.

Original lecture content:
PREDICTIVE PARSING
Homework: left factoring
A → aAB | aBc | aAc
Factor the common initial a.
If alternatives of the new nonterminal still share a prefix, factor again.
Explain how factoring helps predictive parsing.

Original lecture content:
PREDICTIVE PARSING
Predictive parsing roadmap
STEP 3 OF 6
Compute FIRST + FOLLOW
Completed / earlier
Current focus
Next
Check left factoring
Compute FIRST + FOLLOW
Build LL(1) table
FIRST predicts beginnings; FOLLOW predicts what may come next.

Original lecture content:
PREDICTIVE PARSING
FIRST sets: what can begin a string?
A → abc | def | ghi
FIRST(A) = { a, d, g }
FIRST(α) includes ε only if α can derive the empty string.

Original lecture content:

PREDICTIVE PARSING

Computing FIRST correctly

If a is a terminal, FIRST(a) = {a}.

If $A \to \varepsilon$, include ε in FIRST(A).

For $A \to X_1X_2\ldots X_k$, collect FIRST(X_1) excluding ε; continue to X_2 only if X_1 is nullable.

Include ε only when every symbol of that right-hand side can derive ε.

Union the contributions from all productions for A.

Original lecture content:
PREDICTIVE PARSING
Practice: FIRST for expression grammar
E → TE′
E′ → +TE′ | ε
T → FT′
T′ → *FT′ | ε
F → (E) | id
Begin with F, then T′ and E′, then T and E.

Original lecture content:
PREDICTIVE PARSING
Running example — FIRST answer
Set
Value
Why
FIRST(F)
FIRST(T′)
FIRST(T)
FIRST(E′)
FIRST(E)
Do not put ε in FIRST(E) or FIRST(T): neither can derive ε.

Original lecture content:
PREDICTIVE PARSING
FOLLOW sets: what may follow a nonterminal?
FOLLOW(B) contains terminals that can immediately follow B in a sentential form.
Place $ in FOLLOW(S), where S is the start symbol.
FOLLOW never contains ε; $ is allowed.
Unlike FIRST, FOLLOW concerns the context in which a nonterminal appears.
CPT316 • Programming Language Implementation and Paradigms

Original lecture content:
PREDICTIVE PARSING
Computing FOLLOW: three rules
For every production A → α B β:
Add FIRST(β) − {ε} to FOLLOW(B).
If β ⇒* ε, add FOLLOW(A) to FOLLOW(B).
Also put $ in FOLLOW(start symbol).
Repeat until no set changes (set union, never overwrite).

Original lecture content:
PREDICTIVE PARSING
Practice: FOLLOW for expression grammar
E → TE′
E′ → +TE′ | ε
T → FT′
T′ → *FT′ | ε
F → (E) | id
Start here
1. Put $ in FOLLOW(E).
2. F → (E) adds) after E.
3. Propagate through E → TE′ and T →
FT′.

Original lecture content:
PREDICTIVE PARSING
Running example — FOLLOW answer
Set
Value
Main source
FOLLOW(E)
FOLLOW(E′)
FOLLOW(T)
FOLLOW(T′)
FOLLOW(F)
FOLLOW sets accumulate from every relevant occurrence and never contain ε.

Original lecture content:
PREDICTIVE PARSING
Predictive parsing roadmap
STEP 4 OF 6
Build LL(1) table
Completed / earlier
Current focus
Next
Compute FIRST + FOLLOW
Build LL(1) table
Parse input with stack
Use FIRST and FOLLOW to place each production in a table.

Original lecture content:
PREDICTIVE PARSING
Correct algorithm: construct the LL(1) table
For each production A → α:
For every terminal a in FIRST(α) − {ε}, enter A → α in M[A, a].
If ε is in FIRST(α), enter A → α in M[A, b] for every b in FOLLOW(A).
Unused cells indicate syntax errors.
If any cell needs two different productions, the grammar is not LL(1).
Important: write the actual production A → α, not automatically A → ε.
CPT316 • Programming Language Implementation and Paradigms

Diagram representation / semantic relationships:
For each production A -> alpha: for terminals a in FIRST(alpha), add A -> alpha to M[A,a]; if epsilon in FIRST(alpha), add it to M[A,b] for b in FOLLOW(A). Conflicting entries imply non-LL(1).

Original lecture content:
PREDICTIVE PARSING
Step 4: complete a blank LL(1) table
NT / input
id
(
)
*
+
$
E
E′
T
T′
F
Write a production in the appropriate cell based on FIRST, or FOLLOW when nullable.

Original lecture content:
PREDICTIVE PARSING
When a grammar is NOT LL(1)
Original: E → E + T | T
FIRST(E + T) = FIRST(T) = { (, id }
Table conflict
Both alternatives require M[E, id] and M[E, (].
One lookahead token cannot determine the production.
Eliminating left recursion and factoring are useful, but do NOT guarantee that a grammar is LL(1).

Original lecture content:
PREDICTIVE PARSING
Predictive parsing roadmap
STEP 5 OF 6
Parse input with stack
Completed / earlier
Current focus
Next
Build LL(1) table
Parse input with stack
Construct parse tree
Now use the table to accept or reject a sequence of tokens.

Original lecture content:
PREDICTIVE PARSING
Table-driven LL(1) parser: decision rules
Initial stack: $E (top on the RIGHT). Initial input: id + id * id $.
If the top is a terminal, it must match the current input token.
If the top is a nonterminal A, consult M[A, lookahead].
Pop A and push the right-hand side in reverse order (do not push ε).
Accept when both the stack and input are at $.
CPT316 • Programming Language Implementation and Paradigms

Diagram representation / semantic relationships:
LL(1) parser: top of stack terminal -> match lookahead and advance; top nonterminal -> choose production from table; no entry -> error; $ and $ -> accept.

Original lecture content:
PREDICTIVE PARSING
Step 5: parsing trace

Original lecture content:
PREDICTIVE PARSING
Predictive parsing roadmap
STEP 6 OF 6
Construct parse tree
Completed / earlier
Current focus
Next
Parse input with stack
Construct parse tree
Finish
The tree follows the same production choices as the parsing trace.

Original lecture content:
PREDICTIVE PARSING
Step 6: parse tree for id + id * id
CPT316 • Programming Language Implementation and Paradigms

Diagram representation / semantic relationships:
[UNVERIFIED VISUAL RELATIONSHIP: Parse-tree topology for id + id * id must be inspected for every nonterminal and epsilon leaf.]

Original lecture content:
PREDICTIVE PARSING
Leftmost derivation from the parser actions
E ⇒TE′ ⇒FT′E′ ⇒idT′E′ ⇒idE′
⇒id+TE′ ⇒id+FT′E′ ⇒id+idT′E′
⇒id+id*FT′E′ ⇒id+id*idT′E′
⇒id+id*idE′ ⇒id+id*id
A single parse tree has both a leftmost and a rightmost derivation; LL(1) follows the leftmost.

Original lecture content:
PREDICTIVE PARSING
Six-step predictive parsing recap
Remove left recursion
Check left factoring
Compute FIRST +
FOLLOW
Build LL(1) table
Parse input with stack
Construct parse tree
Key check: no conflict in the LL(1) table, and the parsing trace ends with ACCEPT.

Original lecture content:
A BRIEF COMPARISON
BOTTOM-UP PARSING
Shift and reduce from tokens back to the start symbol.

Original lecture content:
BOTTOM-UP PARSING
Bottom-up parsing: shift and reduce
SHIFT
REDUCE
Move the next input token onto the
parsing stack.
Replace symbols matching a
production’s RHS with its LHS.
A successful bottom-up parse reconstructs a rightmost derivation in reverse.
CPT316 • Programming Language Implementation and Paradigms

Diagram representation / semantic relationships:
Bottom-up shift-reduce actions: SHIFT moves next input token to stack; REDUCE replaces a recognized handle with its grammar nonterminal.

Original lecture content:
BOTTOM-UP PARSING
Worked shift-reduce example: id + id
Grammar: E → E + T | T T → id
Stack
Remaining input
Action
$
id + id $
Shift id
$ id
+ id $
Reduce T → id
$ T
+ id $
Reduce E → T
$ E
+ id $
Shift +
$ E +
id $
Shift id
$ E + id
$
Reduce T → id
$ E + T
$
Reduce E → E + T
$ E
$
Accept
CPT316 • Programming Language Implementation and Paradigms

Diagram representation / semantic relationships:
[UNVERIFIED VISUAL RELATIONSHIP: Exact shift/reduce trace row alignment must be verified against source.]

Original lecture content:
BOTTOM-UP PARSING
Top-down vs. bottom-up: final comparison
Property
Top-down / LL(1)
Bottom-up / LR family
Starting point
Start symbol
Input tokens
Main operation
Predict / expand
Shift / reduce
Derivation connection
Leftmost derivation
Reverse rightmost derivation
Typical constraint
Must be LL(1) for 1-lookahead table
Handles a wider range of deterministic
CFGs
Example parsers
Recursive descent, LL(1)
SLR, LALR, canonical LR
Both aim to recognise valid program structure; they differ in how the tree is built.

Original lecture content:
WRAP-UP
Topic 3: what you should now be able to do
Distinguish CFG, linear, right-linear and left-linear grammars.
Draw parse trees and distinguish ambiguity from derivation order.
Eliminate left recursion and apply left factoring correctly.
Calculate FIRST and FOLLOW using set union and ε carefully.
Build an LL(1) parsing table and check for conflicts.
Trace predictive parsing and explain shift-reduce at a conceptual level.
Complete Tutorial 2 and Tutorial 3. Bring any unresolved derivations to class.
CPT316 • Programming Language Implementation and Paradigms